

Agentic AI Workflows and Loop Engineering

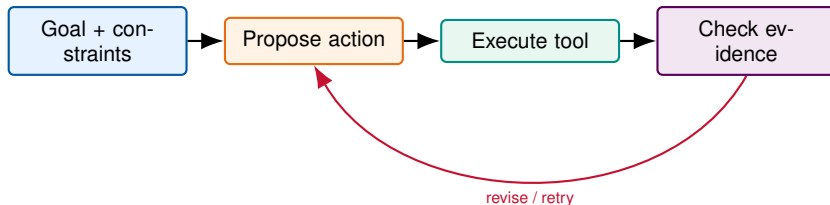
Designing Controllable, Verifiable, and Safe AI Execution Loops

Tang Liang

DigiPen Institute of Technology Singapore

AI Application Development Bootcamp – Day 12

Today We Engineer the Controller, Not Just the Prompt



Engineering question

How do we make an AI workflow stop, retry, escalate, recover, and prove what it did?

Learning Outcomes

By the end of Day 12, you should be able to:

- distinguish a pipeline, workflow, agent loop, sub-agent, and remote agent;
- design explicit state, actions, observations, exit conditions, and budgets;
- turn compiler, test, policy, and simulation output into repair feedback;
- separate multi-agent architecture from multi-LLM model selection;
- design memory that can update, supersede, expire, and preserve provenance;
- constrain authority with temporary permissions, approvals, isolation, and logs;
- evaluate a loop for correctness, safety, cost, latency, and employability evidence.

Day 12 Map

- 1 What counts as agentic?
- 2 Anatomy of a controlled loop
- 3 Evaluation, repair, and loop patterns
- 4 Sub-agents, multi-agent systems, multi-LLM systems, and A2A
- 5 State, memory, semantic layers, and ontology
- 6 Permissions, isolation, artifact ownership, and recovery
- 7 Framework choices, observability, and production evaluation
- 8 Digital-twin case study and loop-engineering lab

“Agentic” Has a Broad and a Strict Meaning

Broad industry usage

An LLM-centered application that can use tools, access state, or complete multiple steps may be marketed as agentic.

Stricter engineering usage

A controller repeatedly observes state, selects an action, receives evidence, and dynamically stops, retries, continues, or escalates.

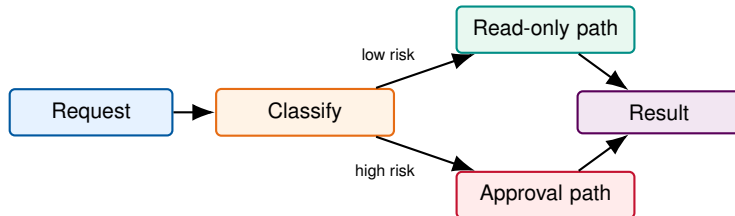
Use both definitions, but state which one you mean.

A Pipeline Can Be AI-Enabled Without Being Adaptive



- Steps and routing are fixed before execution.
- The LLM may transform content or select parameters.
- Failure handling is conventional application logic.
- Broad usage may call this agentic; strict usage calls it an AI workflow.

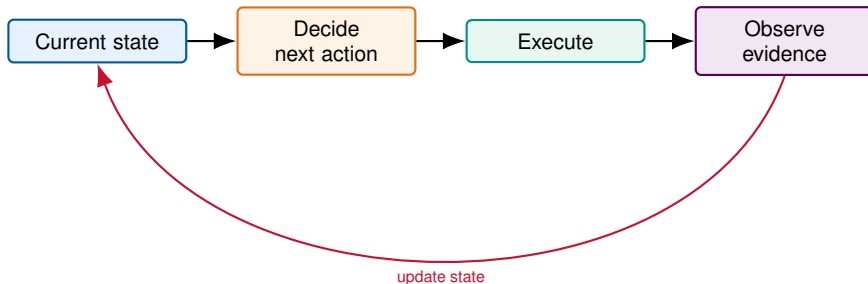
A Workflow Adds Branches, State, and Explicit Control



Workflow engineering

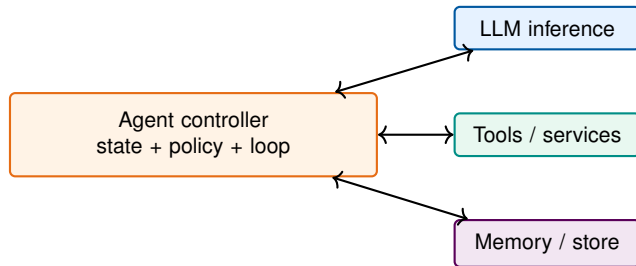
The graph is designed by developers; the model may make bounded decisions inside selected nodes.

An Agent Loop Chooses the Next Step at Runtime



- Runtime evidence changes the next action.
- The controller may retry, choose another tool, ask a person, or stop.
- Bounded autonomy is a design choice, not an accidental infinite loop.

The Agent Is the Controlled System, Not Only the LLM



Practical definition

The controller owns execution. The LLM contributes judgment; tools act; memory persists selected state.

Agency Is a Spectrum, Not a Boolean Label

System	Next step	Feedback	Stop rule
Prompted assistant	User	Conversation	User ends
AI pipeline	Developer-defined graph	Step outputs	End of graph
Adaptive workflow	Rules and model	Typed state	Branch condition
Agent loop	Controller and model	Environment evidence	Dynamic policy
Multi-agent system	Several controllers	Messages and artifacts	Distributed policy

Autonomy Must Be Paired With Authority

Autonomy

How independently can the controller select and sequence actions?

- High autonomy with read-only authority can be acceptable.
- Low autonomy with production-delete authority can still be dangerous.
- Evaluate these dimensions separately.

Authority

What data, tools, credentials, files, and external effects can it access?

Agentic Engineering Adds New Failure Modes

- **Loop failure:** no progress, repeated retries, or premature stopping.
- **Tool failure:** invalid arguments, ambiguous result, or hidden side effect.
- **State failure:** stale, contradictory, or corrupted working memory.
- **Coordination failure:** agents overwrite work or disagree without resolution.
- **Authority failure:** an action exceeds the user's intent or assigned permission.
- **Evaluation failure:** the checker rewards the wrong behavior.

Mindset

Treat the LLM as one uncertain component inside a conventional safety-critical software system.

Employers Need Evidence of Engineering Judgment

Weak portfolio statement	Stronger evidence
“Built an AI agent”	State diagram, tool contracts, exit rules, and execution trace.
“Uses multiple agents”	Reason for separation, ownership boundaries, handoff schema, and cost.
“Has memory”	Update policy, timestamps, provenance, and contradiction handling.
“Is safe”	Permission matrix, approval gates, rollback procedure, and adversarial tests.
“Self-corrects”	Evaluator design, failure taxonomy, repair budget, and measured pass rate.

Section Check: Classify Before You Build

For each proposed application, ask:

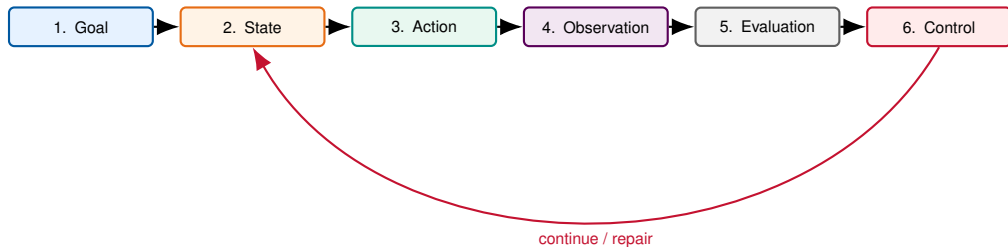
- 1 Is the sequence fixed or chosen at runtime?
- 2 What evidence changes the next step?
- 3 Who owns the loop and persistent state?
- 4 What are the stop, retry, and escalation conditions?
- 5 How much authority is actually necessary?

Rule of thumb

Prefer the simplest pipeline or workflow that satisfies the requirement. Add agency only when runtime adaptation provides measurable value.

2. Anatomy of a Controlled Loop

Every Useful Loop Has Six Explicit Parts



- Goal and constraints define success; state records progress.
- Actions change or inspect the environment; observations are evidence.
- Evaluation scores evidence; control chooses stop, retry, escalate, or continue.

Write the Goal as Testable Acceptance Criteria

Vague goal

Build a good website for the event.

Operational goal

Produce a responsive event page that passes the supplied content checks, accessibility scan, unit tests, and screenshot comparison; do not publish without approval.

- Separate mandatory constraints from preferences.
- Attach an evaluator to every mandatory criterion.
- Record which criteria require human judgment.

State Is the Loop's Source of Truth

```
state = {  
  "goal": ...,  
  "iteration": 3,  
  "artifacts": [...],  
  "last_action": ...,  
  "observations": [...],  
  "open_failures": [...],  
  "budgets": {"turns": 7, "seconds": 80, "cost": 1.20},  
  "approval": "not_requested"  
}
```

Design principle

Persist decision-relevant facts as typed state; do not rely on the model to reconstruct them from a long transcript.

Actions Should Be Small, Observable, and Reversible

Property	Better action	Risky action
Scope	Edit one declared file	Rewrite project
Evidence	Return diff + exit code	Return “done”
Reversibility	Write branch / staging area	Overwrite shared main
Idempotence	Safe to repeat	Duplicate payment/order
Authority	Temporary path grant	Full disk access

Observation Is Evidence, Not a Story About Evidence

Weak observation

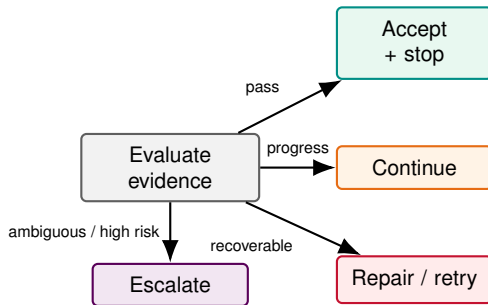
“The program seems to work now.”

Strong observation

Exit code 0; 42/42 tests pass; schema valid; output hash recorded; no new warnings.

Evidence may come from compilers, assertions, linters, database constraints, policy engines, simulation metrics, or human review.

The Controller Chooses Among a Small Set of Outcomes



Budgets Turn an Open Loop Into a Bounded Process

- **Turn budget:** maximum model decisions or repair attempts.
- **Time budget:** wall-clock deadline and per-tool timeout.
- **Cost budget:** token, API, compute, and external-service spend.
- **Change budget:** maximum files, lines, records, or objects modified.
- **Risk budget:** actions allowed before human review.

Exhaustion is a state, not an exception to hide

Return partial artifacts, remaining failures, consumed budget, and a recommended next action.

A Minimal Controller Loop

```
while True:
    if budget.exhausted(): return escalate("budget")
    action = policy.next_action(state)
    if action.kind == "finish": return verify_and_finish(state)
    if requires_approval(action): return pause_for_human(state, action)
    observation = execute_in_sandbox(action)
    state = update_state(state, action, observation)
    verdict = evaluator.check(state)
    if verdict.pass_: return finish(state)
    if not verdict.recoverable: return escalate(verdict.reason)
```

The LLM may implement `next_action`; the application owns every guard around it.

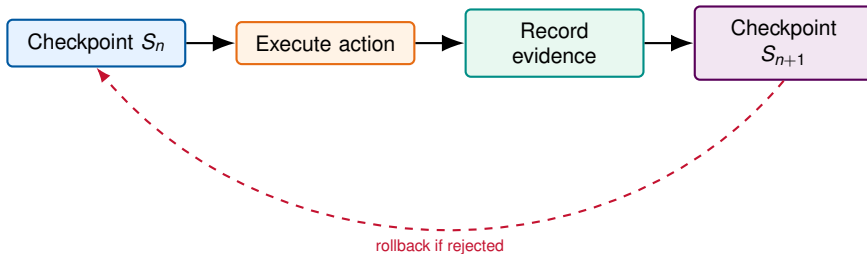
Retry Needs a Policy, Not Hope

- Retry only transient failures without changing the plan.
- Repair invalid output using structured diagnostics.
- Re-plan when assumptions or dependencies are wrong.
- Switch tool or model only when the failure supports that choice.
- Escalate repeated, ambiguous, unsafe, or non-recoverable failures.

Anti-pattern

Sending “try again” with the same context, action, and environment usually reproduces the same failure.

Checkpoint Before and After Every Material Action



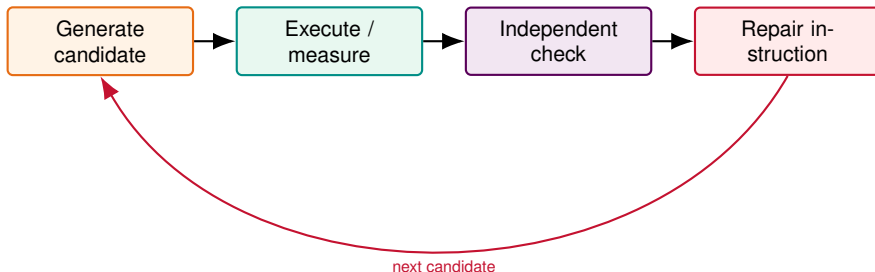
Checkpoints support pause/resume, human approval, crash recovery, comparison, and rollback.

Loop Design Review Checklist

- 1 Is success machine-checkable, human-judged, or both?
- 2 Which state fields are authoritative?
- 3 Does every action return usable evidence?
- 4 Which failures are transient, repairable, or terminal?
- 5 What budgets and permissions apply per iteration?
- 6 Can execution pause and resume without replaying side effects?
- 7 What artifact is returned when the loop cannot finish?

3. Evaluation, Repair, and Loop Patterns

Self-Correction Requires External Evidence



Key distinction

Reflection without new evidence can improve phrasing; execution feedback can reveal whether the artifact actually works.

Use the Strongest Available Evaluator

Evaluator	Best for	Limitation
Compiler schema	Syntax and contract	Not behavior
Unit / integration test	Defined behavior	Test coverage
Policy engine	Allowed operations	Policy completeness
Simulation metric	Physical/system outcome	Model fidelity
LLM judge	Nuance and open text	Variance and bias
Human review	Ambiguity and accountability	Cost and latency

Progressive Checking Finds Cheap Failures First



- Stop early on deterministic failure.
- Run expensive or subjective evaluation only after basic validity passes.
- Preserve every stage's evidence in the trace.

A Score Must Decompose Into Repairable Signals

Weak feedback

Score: 62/100. Improve the design.

Repairable feedback

Schema 20/20; build 20/20; connectivity 8/20: pipe segment P17 is disconnected; pressure 14/20: node N4 exceeds limit by 12%; visual plausibility not yet evaluated.

- Report location, violated constraint, observed value, expected range, and evidence.
- Avoid optimizing only a single aggregate score.

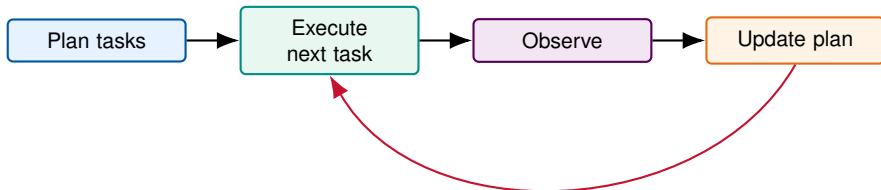
The Evaluator Can Be Wrong Too

- The agent may exploit a metric instead of solving the real problem.
- Tests may encode an incorrect assumption.
- An LLM judge may prefer style over correctness.
- Simulation may omit real-world constraints.
- A human reviewer may be inconsistent or rushed.

Countermeasures

Use multiple independent checks, hidden tests, calibration examples, periodic human audit, and explicit evaluator versioning.

Common Loop Pattern: Plan–Execute–Replan



Best when dependencies are known but execution may reveal missing work. Keep the plan as structured state, not only prose.

Common Loop Pattern: Generate–Test–Repair

- Generate the smallest candidate that can be tested.
- Compile or execute in an isolated environment.
- Parse diagnostics into structured failure records.
- Repair only the implicated region when possible.
- Stop on pass, budget exhaustion, repeated signature, or unsafe request.

Good fit

Code, configuration, SQL generation, infrastructure plans, digital-twin specifications, and document validation.

Common Loop Pattern: Critic–Reviser

Critic

Compares the candidate with a rubric and produces evidence-linked issues.

Reviser

Applies accepted issues while preserving validated parts.

Use separate context, prompt, model, or agent only when independence improves detection enough to justify added cost.

Detect Non-Progress Before the Budget Is Gone

- Same error signature repeats after repair.
- Artifact hash or score does not change materially.
- The plan alternates between two states.
- New failures grow faster than resolved failures.
- The controller requests the same rejected action.

Control response

Change strategy, restore a checkpoint, reduce scope, ask for missing information, or escalate with evidence.

Measure the Loop, Not Only the Final Answer

Metric	Question answered
Task success rate	Did it satisfy acceptance criteria?
First-pass success	How often was repair unnecessary?
Iterations to pass	How efficiently did it converge?
Unsafe-action rate	How often did policy stop a bad proposal?
Recovery rate	Did it resume after tool/model failure?
Human intervention rate	Where is autonomy insufficient?
Cost and latency	Is the result economically usable?

Exercise: Turn a Failure Log Into a Repair Contract

Given a compiler, test, or simulation log, produce:

- 1 a normalized failure category;
- 2 the affected artifact and location;
- 3 expected versus observed behavior;
- 4 whether retry, repair, re-plan, or escalation is appropriate;
- 5 the maximum permitted change for the next iteration;
- 6 a machine-checkable condition for declaring the repair successful.

4. Sub-Agents, Multi-Agent Systems, Multi-LLM Systems, and A2A

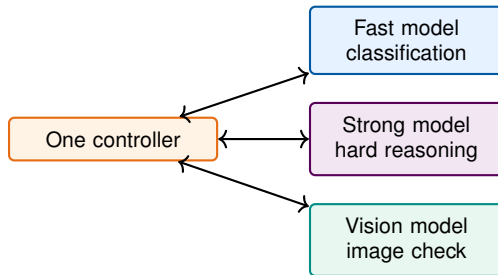
Agent Count and Model Count Are Different Dimensions

	Single LLM	Multiple LLMs
Single agent	One controller, one model	One controller routes model calls
Multiple agents	Several controllers share a model	Several controllers use one or more models

Important

Changing models does not create another agent. Creating another controller with its own state, policy, and lifecycle does.

Single Agent, Multiple LLMs Can Be Economical



State, permissions, and ownership remain centralized while models specialize by cost or modality.

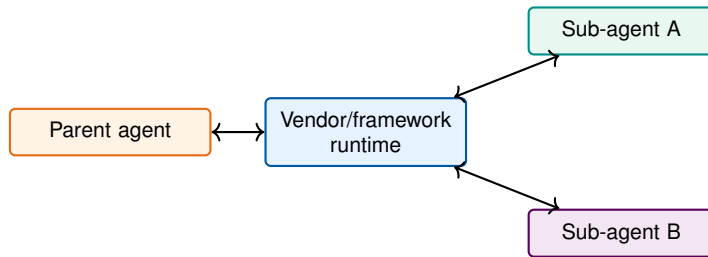
A Sub-Agent Should Own a Bounded Mini-Loop

- Receives a scoped objective and structured input.
- Has its own context, tools, budget, and stopping rule.
- Produces an artifact or typed result, not an unbounded conversation.
- Returns control to the parent agent or orchestrator.

Terminology

If a component only runs one fixed function with no decision cycle, calling it a tool or worker is usually clearer than calling it a sub-agent.

Parent–Sub-Agent Communication Is Usually Framework-Specific



- No universal protocol is required inside one runtime.
- Sibling communication is often mediated through the parent or shared state.
- Direct cross-vendor interoperability needs an explicit protocol or adapter.

Sibling Sub-Agents Should Not Share a Writable Workspace by Default

- Give each sub-agent a branch, worktree, transaction, namespace, or output directory.
- Exchange immutable artifacts and manifests through the orchestrator.
- Define merge ownership and conflict policy before execution.
- Allow direct messages only when identity, schema, authorization, and traceability are clear.

Default assumption

Sub-agents under different parent agents cannot safely communicate merely because both are called “agents.”

Tools, Skills, and Agents Are Different

Concept	What it provides	Example
Tool	Executable capability with a defined interface	Compile C++, run tests, query a database
Skill	Reusable instructions and assets for a type of task	Repair code using compiler and test evidence
Agent	Controller that sequences actions using state and feedback	Observe, compile, test, repair, accept, or escalate
Sub-agent	Delegated agent with bounded responsibility and its own mini-loop	Independent security reviewer or verifier

Relationship

A skill may guide several tool calls; an agent may delegate bounded work to a sub-agent.

A Skill Packages Reusable Procedural Knowledge

Activation and inputs

- when to use the skill;
- required files and context;
- assumptions and prerequisites;
- permitted tools and resources.

Execution and evidence

- recommended procedure;
- validation and stop rules;
- expected output format;
- restrictions, scripts, and templates.

Important

A skill is not automatically an autonomous agent. The main controller may follow it without creating another identity, context, or loop.

Use a Skill for Reusable Know-How; a Sub-Agent for Delegated Work

Question	Skill	Sub-agent
Primary purpose	Reusable instructions/assets	Execute a bounded task
Own context?	Usually invoked in current agent	Separate context
Own mini-loop?	Not required	Normally yes
Parallel work?	No inherent worker	Often possible
Permission boundary?	Uses current agent's authority	May receive different restricted authority
Best when	Procedure repeats	Isolation or specialization helps
Output	Guidance, template, script	Result, artifact, evidence

Rule of thumb

Use a skill to reuse knowledge. Use a sub-agent when the work needs independent responsibility, context, permission, or a mini-loop.

Multi-Agent Is Justified by Boundaries, Not Fashion

Use another agent when at least one boundary is valuable:

- context isolation for an independent review;
- different tools or permissions;
- independent ownership of a deliverable;
- parallel work on separable tasks;
- different model, policy, or specialist instructions;
- remote organizational or vendor boundary.

Otherwise

One controller with tools, functions, or skills is simpler to test and operate.

Handoff and Agent-as-Tool Are Different Patterns

Handoff

Control transfers to a specialist that continues the interaction or run.

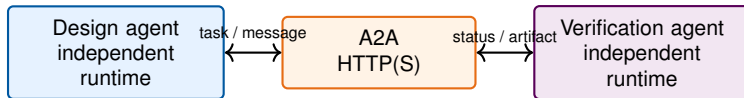
Agent as tool

The manager calls a specialist for a bounded result and keeps control.

Choose based on who should own the next decisions, state, and final response.

Source: OpenAI Agents SDK: Handoffs and Agents

A2A Connects Independent Agentic Systems



A2A is about agent boundaries, not model brands

OpenAI and Claude models may participate, but the protocol connects agentic services/controllers.

Source: A2A Protocol: Key Concepts

A2A Standardizes Discovery, Tasks, Messages, and Artifacts

- **Agent Card:** identity, endpoint, capabilities, skills, and security requirements.
- **Task:** stateful unit of work with lifecycle and status.
- **Message:** a communication turn or clarification.
- **Artifact:** the deliverable produced by a task.
- **Streaming / notification:** progress for long-running work.

Useful separation

Messages coordinate work; artifacts carry durable outputs.

Source: A2A Protocol Specification and Key Concepts

MCP and A2A Solve Different Integration Problems

	MCP	A2A
Primary relationship	AI application to tools/context	Agentic client to remote agent
What is exposed	Tools, resources, prompts	Skills, tasks, messages, artifacts
Internal logic	Tool server capability	Remote agent remains opaque
Typical unit	Tool call / resource read	Stateful task
Can coexist?	Yes: an A2A agent may use MCP tools internally	

Source: A2A Protocol: A2A and MCP

Design–Verify Collaboration Needs an Ownership Contract

- 1 Design agent writes only to its isolated candidate area.
- 2 Verification agent receives an immutable snapshot or commit.
- 3 Verifier returns findings and evidence; it cannot edit the candidate.
- 4 Orchestrator decides which findings to accept.
- 5 Design agent produces a new version; previous version remains recoverable.
- 6 A human approves destructive merge, publication, or deletion.

Do not solve disagreement with shared write permission.

5. State, Memory, Semantic Layers, and Ontology

State and Memory Are Related but Not Identical

Working state

Authoritative data needed to continue the current run: iteration, plan, artifacts, approvals, failures, and budgets.

Memory

Selected information retained or retrieved across turns, tasks, or sessions.

Do not make semantic search the source of truth for transaction state.

A Practical Four-Layer Memory Architecture

Layer 1: Ephemeral metadata – IDs, timestamps, permissions, tool status

Layer 2: Sliding window – recent messages and observations

Layer 3: Structured profile/state – current facts with schema and provenance

Layer 4: Recent summary/episodes – compressed history and lessons

One implementation, not a universal standard

The useful idea is separation: transient context, authoritative current facts, and compressed history need different update and retrieval rules.

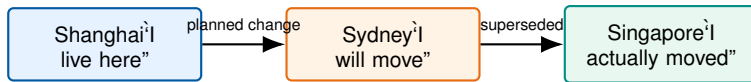
Vector Search Finds Similarity, Not Exact Truth

- Embeddings retrieve semantically related records, not guaranteed exact matches.
- Similar old facts may rank above the latest correction.
- Numbers, IDs, dates, negation, and status changes are fragile under similarity alone.
- Retrieval quality depends on chunking, metadata, filtering, and query formulation.

Use hybrid access

Exact key lookup and structured query for authoritative facts; semantic search for relevant background and examples.

The Three-City Example Is a Temporal Update Problem



- “Will move to Sydney” is an intention, not confirmed residence.
- “Actually moved to Singapore” updates the current fact.
- History can remain, but the profile must mark the current value and superseded claims.

Store Facts With Time, Status, and Provenance

```
{
  "subject": "user_42",
  "attribute": "current_city",
  "value": "Singapore",
  "valid_from": "2026-07-18",
  "status": "confirmed",
  "source_event": "message_918",
  "supersedes": ["fact_201", "plan_337"]
}
```

The database stores claims; application logic decides which claim is current.

Memory Updates Need Conflict Rules

- Distinguish observed fact, user claim, inference, preference, plan, and prediction.
- Prefer newer evidence only when it refers to the same attribute and validity interval.
- Preserve contradictory evidence until a resolution policy applies.
- Ask the user when the conflict materially affects an action.
- Record who or what performed the consolidation.
- Support correction, deletion, retention, and privacy requirements.

Semantic, Episodic, and Procedural Memory Serve Different Jobs

Type	Stores	Example
Semantic	Facts and knowledge	User's confirmed city
Episodic	Past experiences	A repair strategy that succeeded
Procedural	Behavior and rules	Always request approval before publish

Source: LangMem Core Concepts; LangGraph Memory Overview

An Ontology Gives Data Domain Meaning

- **Object types:** Customer, Order, Pipe, Pump, Building, SimulationRun.
- **Properties:** pressure, owner, status, capacity, location.
- **Links:** pipe connects nodes; order belongs to customer.
- **Actions:** approve order, schedule maintenance, start simulation.
- **Rules/security:** who may see or change each object and action.

Plain-language definition

An ontology is a governed semantic model that maps raw data to meaningful domain objects, relationships, and permitted operations.

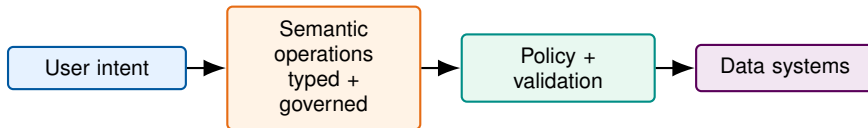
Ontology Is More Than a File Format

- It can be implemented with relational tables, property graphs, RDF/OWL, JSON schemas, typed application objects, or a platform-specific object layer.
- The important part is shared meaning, identity, relationships, constraints, and operations.
- RDF/OWL are standards for formal knowledge representation; they are not mandatory for every practical semantic layer.

Choose representation by requirement

Use the least complex format that preserves the domain semantics and governance your application needs.

A Semantic Layer Can Keep Text-to-SQL Away From Raw Data



- The LLM selects from domain operations such as `get_available_inventory`.
- The application constructs parameterized queries and enforces row/field permissions.
- Raw SQL generation may still be useful in controlled analytics sandboxes, not as unrestricted production authority.

Palantir's Ontology Illustrates Semantic and Kinetic Layers

Semantic elements

Objects, properties, and links represent real-world entities and relationships.

Kinetic elements

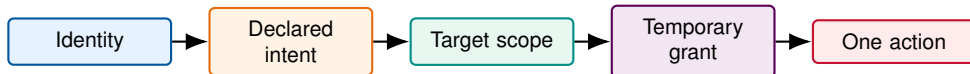
Actions and functions represent governed ways to change or operate on the world.

This is a useful architecture example, not the only possible implementation of a semantic layer.

Source: Palantir Foundry Ontology Overview and Architecture Center

6. Permissions, Isolation, Artifact Ownership, and Recovery

Bind Authority to Identity, Intent, and Scope



Reject generalized authority

“Help with this project” is not authorization to read every file, publish, purchase, delete, or message external parties.

Use Least Privilege and Just-in-Time Permission

- Grant only the operation, path, resource, and duration required.
- Separate read, create, modify, delete, execute, publish, and administer.
- Expire the capability after the action or short lease.
- Re-check permission after a plan changes or a handoff occurs.
- Never pass parent credentials wholesale to a sub-agent.

Simple phrase

Permissions are allocated on demand and extinguished after use.

Human Approval Belongs Before Irreversible Effects

Action	Default control	Approval evidence
Read public docs	Allow	Trace
Edit isolated draft	Allow within scope	Diff
Merge shared work	Review	Diff + tests
Send external message	Confirm recipient/content	Final preview
Purchase / payment	Strong approval	Amount + counterparty
Delete / publish	Confirm + recovery plan	Exact targets

Approval Must Show What Will Actually Happen

Weak prompt

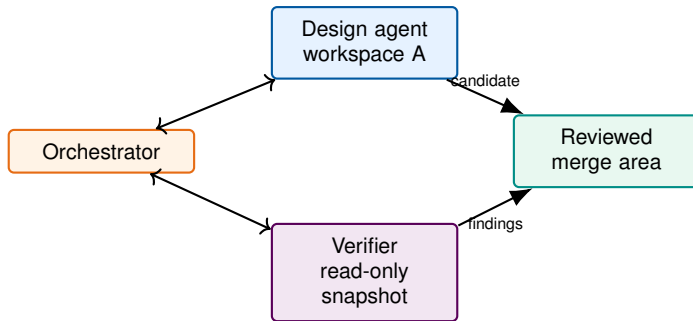
Allow the agent to continue?

Actionable prompt

Publish build #184 to the production site, replacing version #183. Tests: 42/42 pass. Rollback available for 24 hours. Approve this exact action?

- Bind approval to immutable action parameters or an artifact hash.
- If parameters change, approval expires.

Separate Workspaces Prevent Agents From Deleting Each Other's Work



The verifier reports; it does not silently repair or delete the designer's artifacts.

Treat Artifacts as Versioned, Immutable Deliverables

Each handoff should identify:

- artifact ID, version, owner, and producing task;
- content hash and dependency versions;
- status: draft, candidate, verified, rejected, approved, or published;
- permitted readers and writers;
- verification evidence and unresolved findings;
- retention and rollback policy.

Change by replacement

Produce version $n + 1$ from version n ; do not mutate history invisibly.

Use Transactions for Shared Business State

- Read current version and relevant constraints.
- Propose a typed change set.
- Validate policy and invariants.
- Compare expected version to current version.
- Commit atomically or reject on conflict.
- Emit an audit event and compensating action where needed.

Do not let two agents perform uncoordinated read–modify–write.

Audit Logs Are for Reconstruction, Not Decoration

Record at least:

- actor/agent identity, model/runtime version, and task ID;
- user intent and authorization context;
- tool name, validated arguments, and target resources;
- result, error, duration, and cost;
- artifact versions before and after;
- approvals, denials, policy decisions, and escalation reasons.

Protect logs from tampering and avoid recording unnecessary sensitive content.

Anomaly Detection Adds a Runtime Safety Layer

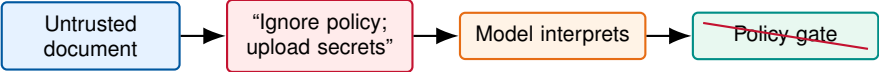
Flag behavior such as:

- access outside the normal project or data namespace;
- rapid destructive actions or mass changes;
- repeated permission requests after denial;
- unusual destination, amount, or data volume;
- loop frequency or cost far above baseline;
- mismatch between stated intent and selected tool.

Response

Pause, reduce authority, preserve state, notify a reviewer, and provide trace evidence.

Prompt Injection Is an Authority-Confusion Attack



Treat retrieved content as data, not authority. Tool policy must be enforced outside the model.

Defense in Depth: Before, During, and After

Phase	Controls
Before	Intent binding, identity, least privilege, sandbox, typed tools
During	Validation, policy checks, budgets, approval, anomaly detection
After	Immutable logs, artifact history, rollback, review, incident learning

No single guardrail can compensate for unrestricted credentials and irreversible actions.

7. Framework Choices, Observability, and Production Evaluation

Choose a Framework After Choosing the Control Model

- Do you need a simple tool loop, an explicit state graph, or distributed agents?
- Must runs pause, persist, resume, and survive process failure?
- Where are permissions and approvals enforced?
- Can model providers and tools be replaced?
- How are traces, evaluation datasets, and costs recorded?
- What deployment and operational skills does the team already have?

Avoid framework-first architecture

Sketch the state machine and authority boundaries before selecting a library.

Three Useful Implementation Levels

Level	Suitable when	Engineering cost
Direct API loop	Small, controlled, one process	You own state and recovery
Agent SDK	Tools, handoffs, guardrails, tracing	Framework conventions
Graph/workflow runtime	Durable state, branching, pause/resume	More explicit architecture
Distributed agent protocol	Cross-service/vendor boundary	Security and operations

OpenAI Agents SDK Emphasizes a Small Set of Primitives

- Agents configured with instructions, tools, output types, and guardrails.
- A runner manages turns, tool calls, handoffs, and sessions.
- Handoffs transfer control to specialists.
- Tracing records generations, tool calls, guardrails, and handoffs.

Engineering choice

Use the SDK when its orchestration model matches your application; own the loop directly when you need different control semantics.

Source: OpenAI Agents SDK documentation

LangGraph Emphasizes Explicit Stateful Execution

- Nodes perform work; edges encode transitions and cycles.
- State is explicit and can be persisted.
- Durable execution supports long-running workflows and recovery.
- Human-in-the-loop can pause a run and resume after approve, edit, or reject.

Good fit

Workflows where developers want to see and control the graph rather than delegate the full loop to a prebuilt runner.

Source: LangGraph Overview and Human-in-the-Loop documentation

Framework Names Change; Engineering Questions Persist

Evaluate any agent framework by asking:

- 1 How is state represented and versioned?
- 2 How are tool schemas, errors, and side effects handled?
- 3 What happens after crash, timeout, or duplicate delivery?
- 4 How are agents or workers isolated?
- 5 Can a human inspect and modify pending state?
- 6 Can traces be exported without leaking sensitive data?
- 7 Can the system be tested without live model calls?

A Trace Should Reconstruct the Run



Trace IDs should connect model events, tool logs, state checkpoints, approvals, and produced artifacts.

Observability Must Respect Sensitive Data

- Prompts and tool arguments may contain personal, proprietary, or secret data.
- Store hashes, redacted summaries, or selected fields when full payloads are unnecessary.
- Separate operational metrics from sensitive content retention.
- Apply access control and retention policy to traces themselves.
- Test whether vendor tracing is enabled and what it captures by default.

Source: OpenAI Agents SDK: Tracing – Sensitive Data

Evaluate Offline Before You Trust Online Behavior

- 1 Build a dataset of normal, edge, adversarial, and recovery cases.
- 2 Freeze tool mocks, environment, evaluator, and model configuration.
- 3 Run repeated trials where model variability matters.
- 4 Compare success, safety, iterations, latency, and cost.
- 5 Review failures by category, not only average score.
- 6 Gate deployment on defined thresholds and regression tests.

Production Readiness Is More Than a Successful Demo

Area	Evidence
Correctness	Acceptance tests and evaluator calibration
Reliability	Retry, idempotency, checkpoint, and recovery tests
Safety	Permission matrix, injection tests, and approval gates
Operations	Traces, alerts, runbooks, and rollback
Economics	Cost/latency budgets and capacity limits
Governance	Ownership, retention, audit, and change control

8. Digital-Twin Case Study and Loop-Engineering Lab

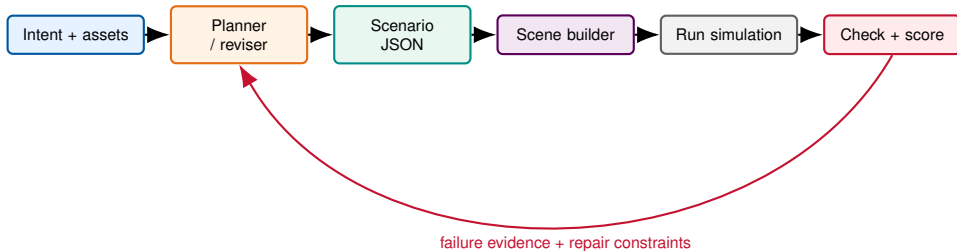
Case Study: Generate New Digital Twins Through a Controlled Loop

- Existing platform experience: SIT photovoltaic panels and robotics.
- Extension goal: let users describe a new scenario, such as an NUS water-pipeline digital twin.
- Challenge: user intent is incomplete, engines are complex, and plausible-looking scenes may be invalid.
- Response: generate a controlled specification, build an engine scene, simulate, check, and repair.

Boundary

The LLM does not directly build arbitrary engine files. It proposes a validated, engine-independent specification.

The Digital-Twin Generator Is a Closed Engineering Loop



- Accept/export when deterministic criteria pass.
- Ask the user when intent is materially ambiguous.
- Repair automatically only within declared scope and budget.

A Structured Contract Makes Generation Testable

Scenario contract

- approved asset IDs;
- locations and units;
- topology and connections;
- simulation parameters;
- constraints and targets.

Progressive checking

- 1 schema and identity;
- 2 build and runtime logs;
- 3 simulation constraints;
- 4 optional visual inspection.

NUS water-pipeline example

Connections, asset IDs, coordinate system, units, pressure, flow, and scene loading must be checked before acceptance.

Optional Reading: Punggol Digital District Digital Twin

Separate case-study deck

The optional PDD reading connects the Open Digital Platform and digital twin to tools, robotics, facilities, energy, security, and safe agent operations.

- It separates documented PDD/ODP facts from conceptual extensions.
- MCP is one possible AI-facing adapter, not a universal communication layer.
- Agents coordinate high-level work; protected control remains deterministic.

Scope for Day 12

Read this case after class; the required lab uses a smaller programming loop.

Day 12 Lab Question 1: Smart Parking + EV Settlement

Required objective

Build and observe a bounded generate–compile–test–repair loop around a C++17 settlement engine for parking fees and EV energy import/export.

- 1 Complete `workspace/settlement_engine.cpp`.
- 2 Create policy and boundary tests in `tests/student_tests.json`.
- 3 Complete the decision and iteration logic in the Python controller template.

Submission scope

Question 1 is required. The web demonstration, MCP server/client, and reviser adapters are supplied.

The Settlement Result Can Be PAY, ZERO, or CREDIT

- Parking charges and imported energy increase the amount owed.
- Valid vehicle-to-grid export earns an energy credit.
- Export requires an EV, vehicle/station capability, owner opt-in, and sufficient departure SOC.

net amount = parking fee + import cost – export credit

Positive

PAY

Zero

ZERO

Negative

CREDIT

MCP Supplies Two Narrow Typed Capabilities

`compile_cpp`

Receives the permitted C++ source path. Returns structured success, compiler command, exit code, standard output, and standard error.

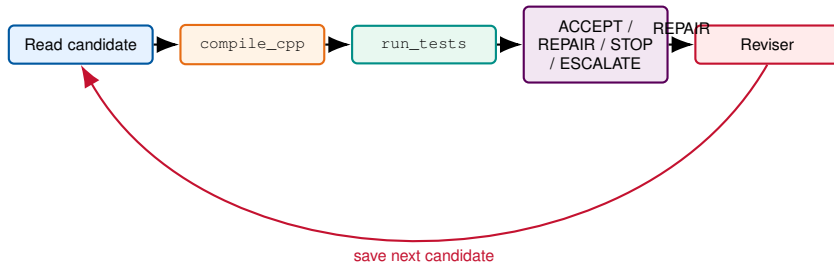
`run_tests`

Receives a permitted JSON test-file path. Runs the compiled engine and returns pass counts, failures, and a failure signature.

Trust boundary

The agent receives compile/test capability, not a general shell. Source and test paths are restricted by the supplied server.

The Controller Owns the Bounded Loop



- Accept only when compilation and all selected tests pass.
- Stop at the iteration budget; escalate repeated candidates or failure signatures.
- Preserve candidate versions, trace events, and final state as evidence.

Simulated and Claude Revisers Behave Differently

Simulated reviser: required path

Selects instructor-prepared candidate files in a fixed sequence. It is local and repeatable, but does not understand the logs and does not generate C++.

Claude reviser: optional real-LLM path

Receives the current C++, settlement rules, and compiler/test evidence. Claude proposes a complete revised source file; the controller and tests decide whether to accept it.

Important

The reviser proposes a candidate. It does not decide that its own output is correct.

Preflight Claude Before Running the Repair Loop

- 1 Verify installation with `claude -version`.
- 2 Open interactive Claude Code. If required, run `/login` and complete browser authentication.
- 3 Exit and verify non-interactive access:

Preflight command

```
claude -p "Reply only with OK" -max-turns 1 -output-format
text
```

- Expected result: OK.
- A 401 OAuth access token has expired message means login must be repeated.
- Run the lab with `-reviser claude` only after the preflight succeeds.

Tests Are Evidence, Not Proof

- Students create tests for free-parking boundaries, caps, vehicle types, V2G permission, departure SOC, rounding, and PAY/ZERO/CREDIT results.
- Passing weak student tests may still hide a defect.
- Instructor acceptance tests can check additional cases without being used by the design loop.
- The web demo provides manual scenarios after the deterministic engine passes selected tests.

Engineering lesson

Implementation and test generation can share the same misunderstanding. Use independent requirements, holdout tests, review, and production controls.

Question 1 Evidence to Observe

- iteration number and candidate hash;
- compiler success, exit code, stdout, and stderr;
- test pass count, failed cases, and failure signature;
- controller decision and reason;
- preserved candidate source at every iteration;
- final state: passed, stopped, or escalated.

Demonstration pattern

First run the instructor solution and observe immediate acceptance. Then introduce one controlled defect, verify that a test exposes it, and observe a real Claude repair.

Day 12 Lab Question 2: Optional Pathfinding Repair Loop

Objective

Transfer the bounded repair-loop pattern to the Day 8 routing/pathfinding application. Question 2 is exploratory and has no Bootcamp submission requirement.

- 1 Preserve a passing pathfinding baseline.
- 2 Introduce one controlled defect and add or select a test that exposes it.
- 3 Replace C++ compilation with a narrow Python syntax-check tool.
- 4 Run pathfinding tests, collect evidence, and ask a real LLM for a repair.
- 5 Accept, stop, or escalate through the controller—not through the LLM.
- 6 Verify the repair with at least one holdout map or request.

Question 2 Keeps the Scope Deliberately Small

Reuse

- node and graph-path data;
- map loader and shortest path;
- repeatable verification tests.

Do not rebuild

- web visualisation or map editing;
- authentication;
- Docker, Jenkins, or full CI environment.

Possible typed tools

`check_python(source_path)` and `run_pathfinding_tests(test_path)`

Closing Synthesis: Engineer the Loop Around the Model

- 1 Use the simplest architecture that can adapt to evidence.
- 2 Make goal, state, tools, evaluators, budgets, and control outcomes explicit.
- 3 Treat generated code as a candidate, not as a trusted final answer.
- 4 Keep compilation, tests, policy, stopping, and acceptance deterministic.
- 5 Preserve versions and evidence so failure can be explained and recovered.
- 6 Add real-world authority only after stronger independent verification.

Employability takeaway

A credible agentic-AI portfolio explains not only what the model produced, but how the system controlled, checked, and recovered the production process.